Models & System Design

1. Models Used

Language Models (LLMs)

Model	Provider	Used For	How It's Called
`meta/llama-3.1-8b-instruct`	NVIDIA AI Endpoints	FAQ answer generation, general chat with tool-calling, memory evaluation, short-term memory summarization	`langchain_nvidia_ai_endpoints.ChatNVIDIA`
`llama-3.1-8b-instant`	Groq	Low-latency streaming responses in live audio/video chat, auto-generating session titles	`langchain_groq.ChatGroq` (with `streaming=True`)
`openai/gpt-4o-mini`	OpenRouter	Vision analysis of camera frames during live video chat	`langchain_openai.ChatOpenAI` with `base_url="https://openrouter.ai/api/v1"`
`meta/llama-3.2-11b-vision-instruct`	NVIDIA AI Endpoints	Static image upload analysis (`/api/vision` endpoint)	`ChatNVIDIA` with multimodal message content

Embedding & Reranking Models (Local)

Model	Library	Used For
`all-MiniLM-L6-v2`	`sentence-transformers`	Encodes user questions and PDF chunks into 384-dim vectors for Neo4j and Qdrant similarity search
`cross-encoder/ms-marco-MiniLM-L-6-v2`	`sentence-transformers` (CrossEncoder)	Reranks all retrieved candidates by relevance score before routing and generation

Both models are loaded at server startup and held in memory for the lifetime of the process.

Speech Models (Sarvam AI)

Model	Direction	Used For
`saaras:v3`	Speech → Text	Transcribing voice input (REST file upload, WebSocket streaming, live video chat). Language: `en-IN`, mode: `codemix` (Hinglish)
`bulbul:v3`	Text → Speech	Generating spoken responses. Speaker: `shubh`, Language: `hi-IN`. Supports both batch and streaming (sentence-boundary parallel fetching)

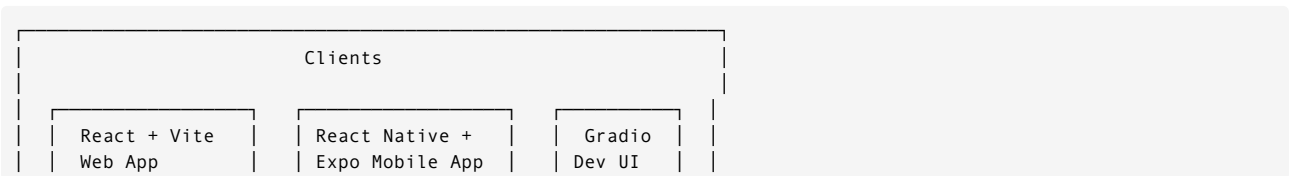
Voice Activity Detection (VAD)

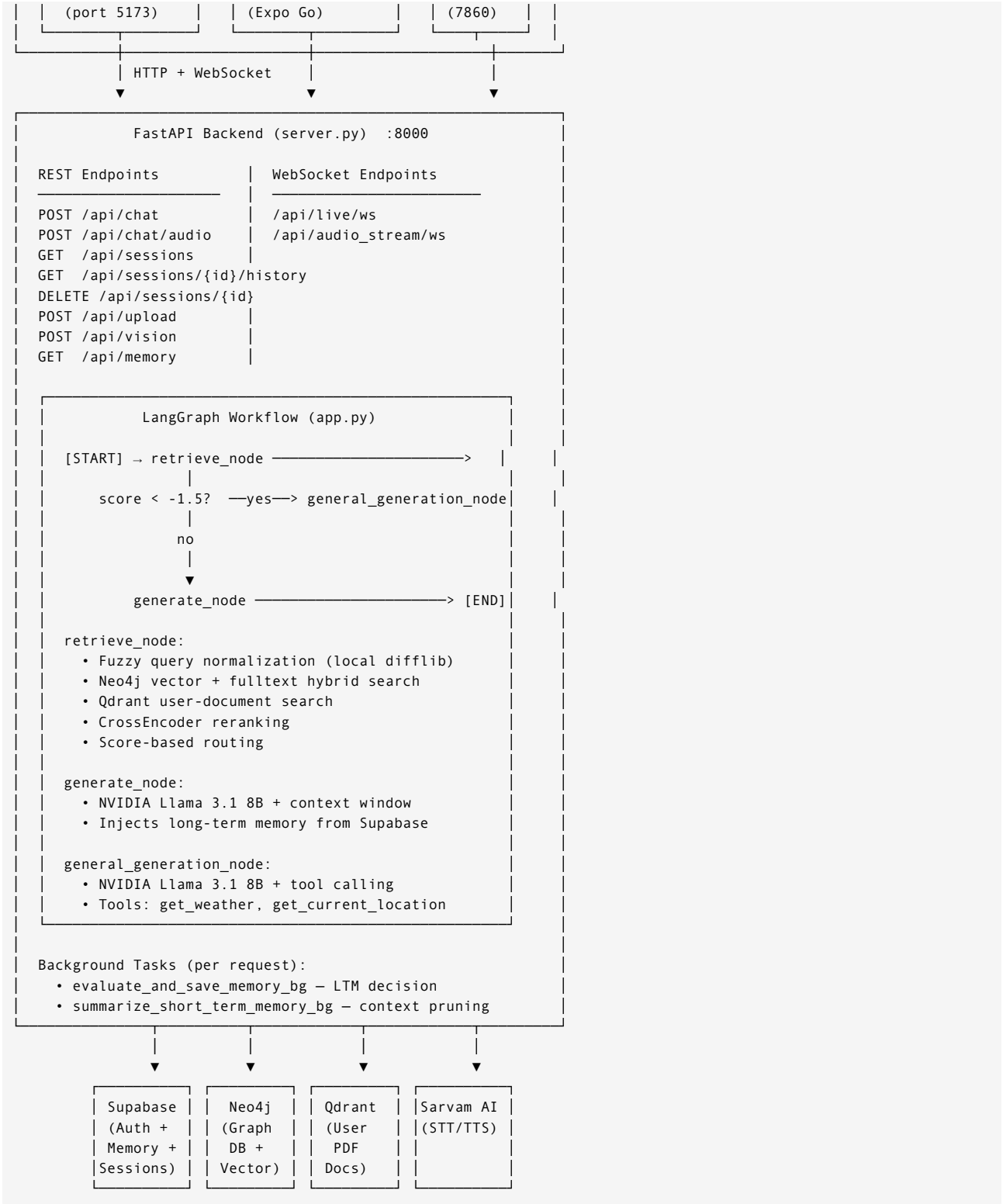
Model	Library	Used For
Silero VAD	`silero_vad` (PyTorch)	Detects when the user starts and stops speaking. Runs on 512-sample (32ms) windows of 16kHz PCM audio. Threshold: 0.5, Min silence: 1000ms

Silero VAD is loaded once (`load_silero_vad()`) and reused across all connections. A `VADIterator` instance is created per session.

2. System Design

High-Level Architecture





Data Flow: Text Chat Request

```
User types message
      |
      ▼
React App.jsx → POST /api/chat (Bearer token)
      |
      ▼
server.py → Verifies JWT via supabase.auth.get_user()
      |
      ▼
Auto-create session title (Groq, if new session)
      |
```

```
      ▼
LangGraph.ainvoke(initial_state) with thread_id=session_id
      |
      ├─ retrieve_node ──────────────────────────────────────┐
      |      • Normalize query                               |
      |      • Encode → 384-dim vector                       |
      |      • Neo4j: vector search + fulltext search        |
      |      • Qdrant: user-scoped document search           |
      |      • CrossEncoder reranking + routing              |
      |                                                      |
      ├─ (router=1) → general_generation_node ──────────────┤
      |      • NVIDIA LLM + weather/location tools           |
      |      • A2UI JSON parsing                             |
      |                                                      |
      └─ (router=2) → generate_node ─────────────────────────┘
            • Fetch LTM from Supabase
            • NVIDIA LLM + retrieved context
      |
      ▼
process_a2ui_messages() — parse and strip any JSON cards
      |
      ▼
Background: evaluate_and_save_memory_bg()
Background: summarize_short_term_memory_bg()
      |
      ▼
Response: { text, audio_base64, a2ui_messages, surface_id }
```

Data Flow: Live Video Chat (WebSocket)

```
Browser                    Server
   |                          |
   ├── auth { token, sid } ──>| → verify JWT, create Session
   |                          |
   ├── audio_chunk (b64 PCM) ─>| → append to VAD buffer
   |                          |   → run Silero VAD per 512-sample window
   |                          |   → open StreamingSTTSession on speech start
   |                          |   → finalize STT on speech end
   |                          |   ← partial_transcript (live)
   |                          |   ← transcript (final)
   |                          |
   ├── video_frame (b64 JPEG) >| → push to frame_buffer (maxlen=5)
   |                          |   → scene change detection (MD5 hash)
   |                          |
   |                          | handle_user_question():
   |                          |   → needs_visual_context()? yes/no
   |                          |   → if yes: compress frame → Vision LLM
   |                          |           → send TTS filler
   |                          |   → LLM producer: astream tokens
   |                          |   ← text_chunk (per token)
   |                          |   → TTS consumer: fetch Sarvam per sentence
   |                          |   ← tts_chunk (per sentence)
   |                          |   ← assistant_response (final)
   |                          |
   ├── audio_chunk (during TTS>| → barge-in detected
   |   playback)              |   ← interrupt_ack
   |                          |   → stop LLM stream + flush TTS queue
```

Data Store Responsibilities

Store	Technology	Data
Auth + Sessions + Memory	Supabase (PostgreSQL)	User accounts (JWT auth), `chat_sessions` table, `user_memories` table
FAQ Knowledge Graph	Neo4j	`Topic → Subtopic → FAQ` nodes, vector index (`faq_embeddings`), Lucene fulltext index (`faq_text_index`)
User Documents	Qdrant Cloud	PDF text chunks with 384-dim embeddings, filtered by `user_id` and `session_id`
Conversation State	SQLite (`checkpoints.db`)	LangGraph message checkpoints per `thread_id` (session)
Live Session State		Per-WebSocket session objects (frame buffer, conversation history, VAD state)

Store	Technology	Data
	In-memory (`ConnectionManager`)	

FAQ Data Pipeline

```
data/jio_faq_data.json (200+ FAQs)
        │
        ▼
load_to_graph.py → Neo4j: Topic → Subtopic → FAQ nodes
        │
        ▼
embed_faqs.py → all-MiniLM-L6-v2 → Neo4j vector index
        │
        ▼
create_index.py → Lucene fulltext index on question + answer fields
```

PDF Ingestion Pipeline

```
User uploads PDF
        │
        ▼
Docling (converter_pdf) → Markdown text
        │
        ▼
RecursiveCharacterTextSplitter (1000 chars, 200 overlap)
        │
        ▼
SentenceTransformer batch encode → 384-dim vectors
        │
        ▼
Qdrant upsert (jio_documents collection)
    payload: { user_id, session_id, text, document_id }
```

Memory Architecture

```
                    ┌──────────────────────────┐
                    │      Memory Tiers        │
                    └──────────────────────────┘

┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│  Long-Term (LTM) │  │ Short-Term (STM) │  │  Session State   │
│                  │  │                  │  │                  │
│ Supabase         │  │ SQLite           │  │ In-memory        │
│ user_memories    │  │ checkpoints.db   │  │ ConnectionManager│
│                  │  │                  │  │                  │
│ Cross-session    │  │ Per session      │  │ Per WebSocket    │
│ user facts       │  │ message history  │  │ live chat state  │
│                  │  │                  │  │                  │
│ Permanent        │  │ Auto-summarized  │  │ 600s TTL         │
│ (until cleared)  │  │ at >5 messages   │  │ lost on restart  │
└──────────────────┘  └──────────────────┘  └──────────────────┘
```
